PAYPAL

Project overview:

At PayPal, I worked as a Data Scientist on a fraud detection and transaction risk scoring project focused on identifying suspicious payment activities in real time and helping fraud analysts make faster and more accurate decisions. The core objective of the project was to reduce fraudulent transactions while ensuring genuine customer payments were not impacted. I worked on large-scale payment and behavioral datasets containing millions of transactions per day.

I built and optimized machine learning models using Python and SQL to score transactions based on risk signals such as transaction velocity, device behavior, customer history, and merchant patterns. By improving feature engineering and model tuning, the fraud recall improved by around 12 to 18 percent, which helped the business catch more fraud cases without increasing false positives.

In addition to traditional ML models, I worked on a Generative AI–based fraud explanation system using AWS Bedrock with GPT-style large language models. The goal was to convert complex fraud model outputs into simple, human-readable explanations for fraud analysts and compliance teams. I designed structured prompt-engineering pipelines in Python, where transaction details, customer behavior aggregates, and model scores were dynamically injected into prompts. This reduced unclear explanations and lowered analyst clarification requests by about 22 percent.

For data processing and explainability, I built an SQL-based data layer on Snowflake that aggregated high-volume transaction and behavioral features into inference-ready tables, ensuring low-latency access and meeting millisecond-level SLA requirements. I also implemented evaluation and hallucination-control metrics using MLflow, which reduced misleading or incorrect explanations by nearly 30 percent during A/B testing.

The solution was deployed on AWS using Lambda and API Gateway to deliver near real-time explanations at scale. Overall, the project reduced fraud investigation time by 25 to 35 percent and improved decision accuracy for fraud analysts.

Detailed Step by step Explanation of your Project

Step 1: Understanding the Fraud Problem and Exploring Large-Scale Payment Data

The first step of the project was to clearly understand the fraud problem PayPal was trying to solve. The goal was to detect fraudulent transactions in real time while ensuring genuine customer payments were not wrongly blocked. I worked with very large payment and behavioral datasets that processed millions of transactions per day. These datasets included transaction amounts, timestamps, merchant details, device information, customer behavior history, and payment velocity patterns.

Using SQL, I explored historical transaction data stored in Snowflake to understand fraud trends, common fraud patterns, and high-risk transaction behaviors. I analyzed fraud rates across geographies, devices, and merchants to identify signals that were most predictive of fraud. This exploration helped identify features such as rapid transaction attempts, unusual device usage, abnormal spending behavior, and account age patterns.

I used Python (Pandas and NumPy) to clean the data, handle missing values, normalize numeric features, and encode categorical variables. During this phase, I worked closely with Fraud Operations teams to understand how fraud analysts interpret suspicious activity and what information they need to make accurate decisions.

This alignment was critical to ensure that the models did not just perform well statistically but were also practical for real-world fraud review.

This step laid the foundation for the entire project by ensuring the data was reliable, meaningful, and aligned with business and compliance requirements. It also helped define success metrics such as improving fraud recall without negatively impacting payment authorization latency.

Step 2: Building and Optimizing Machine Learning Fraud Detection Models

After preparing the data, I built and optimized machine learning fraud detection and transaction risk scoring models using Python, SQL, and AWS. The objective of these models was to assign a fraud risk score to each transaction in near real time. I experimented with multiple algorithms such as logistic regression, gradient-boosted trees, and ensemble models to balance performance and interpretability.

Feature engineering played a major role in improving model performance. I created behavior-based features such as transaction velocity, average spending deviation, device reuse frequency, and merchant risk scores. These features were computed using SQL on Snowflake, aggregating high-volume transaction data efficiently. I then used Python to train models and tune hyperparameters using cross-validation.

Through continuous optimization and evaluation, the fraud recall improved by 12–18 percent, meaning the system was able to catch significantly more fraudulent transactions without increasing false positives. This improvement had a direct business impact by reducing financial losses and improving customer trust.

The trained models were deployed on AWS, ensuring scalability and low latency. I monitored model performance over time to detect drift and performance degradation. This step ensured that fraud detection remained accurate even as fraud patterns evolved. The output of these models later became a critical input for the Generative AI explanation system.

Step 3: Designing and Implementing the Generative AI–Based Fraud Explanation Framework

After building and stabilizing the fraud detection models, the next major step was to design a Generative AI–based fraud explanation framework. The purpose of this step was to convert complex machine learning outputs into clear, human-readable explanations that fraud analysts and compliance teams could easily understand. I used AWS Bedrock with GPT-style large language models for this task.

I created structured prompt-engineering pipelines using Python. Each prompt was dynamically populated with real transaction data such as fraud risk scores, transaction velocity, device indicators, customer behavior aggregates, and merchant risk signals. This ensured that the LLM explanations were always grounded in actual data and model outputs rather than generic text. For example, instead of producing vague explanations, the system clearly stated why a transaction was risky, such as unusual spending behavior or rapid transactions from a new device.

I carefully designed prompt templates to control tone, length, and consistency across millions of transactions. This reduced ambiguous or inconsistent explanations and improved explanation clarity. As a result, fraud analyst clarification requests reduced by 22 percent, showing that the explanations were easier to understand and more actionable.

This framework also supported regulatory explainability requirements, which are critical in financial services. By translating technical fraud signals into plain language, the solution improved analyst decision accuracy by 18 percent, helping teams make faster and more confident fraud decisions.

Step 4: Building the Explainability Data Layer and Optimizing for Low Latency

To support real-time fraud explanations, I built a dedicated explainability data layer using SQL on Snowflake. This layer aggregated high-cardinality transaction, merchant, and behavioral features into inference-ready tables that could be accessed quickly by both machine learning models and LLM pipelines.

I designed optimized SQL queries to pre-compute frequently used features such as transaction velocity metrics, customer behavior summaries, and merchant risk indicators. By using partitioning, indexing, and materialized views, I ensured that the system could handle millions of transactions per day while maintaining millisecond-level SLA compliance. This was critical because fraud explanations had to be generated without impacting payment authorization latency.

This data layer acted as a single, consistent source of truth for fraud scoring and explanations. It ensured that the explanations shown to analysts were always aligned with the underlying fraud model outputs. Low-latency access enabled near real-time LLM inference, allowing explanations to be generated instantly during fraud reviews.

By optimizing data access and reducing runtime computations, this step played a key role in scaling the solution efficiently. It allowed PayPal to deploy fraud explanations at scale without adding performance overhead to the payment system.

Step 5: Evaluation, Deployment, and Measuring Business Impact

In the final phase of the project, I focused on evaluation, deployment, and measuring business impact. I implemented evaluation and hallucination-control metrics using MLflow to track explanation accuracy, consistency, and alignment with the original fraud models. These metrics helped ensure that LLM outputs remained factual and did not introduce misleading information. I conducted controlled A/B testing to compare traditional fraud explanations with LLM-generated explanations. Based on these tests, incorrect or misleading explanations were reduced by 30 percent, and analyst trust in the system increased. This evaluation process was essential to safely deploy Generative AI in a regulated financial environment.

The final solution was deployed using AWS Lambda and API Gateway, enabling scalable and near real-time LLM inference. The system supported millions of transactions per day without affecting transaction latency. I worked closely with Fraud Operations and Compliance teams to integrate explanations into analyst dashboards using Tableau. As a result, average fraud investigation time reduced by 25–35 percent, and customer dispute resolution turnaround improved significantly. This step demonstrated the real business value of combining machine learning, Generative AI, and scalable cloud infrastructure.

Important Interview Questions & Answers

1. How did you improve fraud recall by 12–18% without increasing false positives?

Improving fraud recall without increasing false positives was one of the most important challenges in this project. I focused mainly on feature engineering, model tuning, and threshold optimization. First, I analyzed historical fraud cases to understand which behavioral signals were most predictive. Using SQL on Snowflake, I created features such as transaction velocity, spending deviation from a user's normal behavior, device reuse frequency, and merchant risk indicators. Next, I trained multiple models in Python, including logistic

regression and gradient-boosted tree models, and compared their performance using precision-recall curves rather than accuracy. Since fraud data is highly imbalanced, recall was prioritized, but precision was carefully monitored. I tuned hyperparameters using cross-validation to avoid overfitting.

Instead of using a fixed decision threshold, I optimized thresholds based on business risk tolerance. This allowed the system to catch more fraud cases while keeping false positives stable. I also continuously monitored performance after deployment to detect drift. By combining better features, careful evaluation metrics, and threshold tuning, the fraud recall improved by 12–18% while maintaining a consistent customer experience.

2. How did you design the Generative AI fraud explanation system using AWS Bedrock?

The Generative AI fraud explanation system was designed to translate complex fraud model outputs into simple, human-readable explanations. I used AWS Bedrock with GPT-style LLMs as the core engine. The key challenge was ensuring explanations were accurate, consistent, and compliant. I built structured prompt-engineering pipelines in Python. Each prompt dynamically included transaction metadata such as fraud risk score, transaction velocity, device indicators, and customer behavior summaries. This grounding ensured the LLM generated explanations strictly based on real data.

I used prompt templates with fixed structure, controlled tone, and limited output length to avoid hallucinations. I also added explicit instructions telling the model to explain only the strongest fraud signals. This approach reduced vague explanations and improved clarity. The system reduced analyst clarification requests by 22% and improved decision accuracy by 18%, showing that the explanations were actionable and trustworthy. This framework also supported regulatory explainability requirements.

3. How did you prevent hallucinations in LLM-generated fraud explanations? (Expanded)

Preventing hallucinations in LLM-generated fraud explanations was a critical requirement because incorrect explanations could mislead fraud analysts and create regulatory risk. I approached this problem using a combination of data grounding, prompt control, evaluation metrics, and testing.

First, I ensured strong data grounding. Every LLM prompt was populated only with verified inputs such as fraud model scores, transaction velocity features, device indicators, customer behavior aggregates, and merchant risk signals. These inputs were pre-computed using SQL on Snowflake, so the LLM never had to guess or infer missing information. The prompt explicitly instructed the model to explain only the provided signals and not introduce new reasons.

Second, I applied strict prompt-engineering controls using Python. I used fixed prompt templates with clearly defined sections such as “Key Risk Signals” and “Explanation Summary.” I limited response length and restricted language flexibility so that explanations remained factual and consistent across transactions. This significantly reduced vague or creative outputs.

Third, I implemented hallucination-control and alignment metrics using MLflow. I tracked how often explanations aligned with underlying fraud scores and features. During controlled A/B testing, LLM explanations were compared with baseline rule-based explanations. Any mismatch or misleading explanation was flagged and analyzed.

By iteratively refining prompts and evaluation rules, incorrect or misleading explanations were reduced by 30%. This made the system reliable, compliant, and safe to deploy in a production financial environment.

4. Why did you use Snowflake for the explainability data layer?
Snowflake was chosen for the explainability data layer because it could efficiently handle large-scale, high-cardinality transaction data while meeting strict latency requirements. In this project, the system processed millions of transactions per day, so fast and reliable data access was essential.

I used SQL on Snowflake to build inference-ready tables that aggregated transaction, customer, merchant, and behavioral features. These included transaction velocity metrics, historical spending patterns, device reuse indicators, and merchant risk summaries. Instead of calculating these features during real-time inference, I pre-computed them using optimized SQL queries.

To improve performance, I applied partitioning and indexing strategies and created materialized views for frequently accessed features. This allowed the fraud models and LLM pipelines to fetch data in milliseconds, supporting millisecond-level SLA compliance. This was especially important because fraud explanations had to be generated without delaying payment authorization.

Snowflake also acted as a single source of truth for both fraud scoring and explanations. This ensured consistency—what the model used to score a transaction was exactly what the LLM used to explain it. This alignment improved trust among fraud analysts and compliance teams.

Overall, Snowflake enabled scalability, low latency, and consistency, making it a strong foundation for real-time explainable fraud detection.

5. How did you evaluate ML models and LLM explanations together?

Evaluating traditional machine learning models and LLM explanations together required a multi-layered approach. For fraud detection models, I focused on metrics such as recall, precision, and precision-recall curves, because fraud data is highly imbalanced. The main business goal was to improve recall, which increased by 12–18%, while keeping false positives under control.

For LLM-generated explanations, I introduced additional evaluation dimensions. Using MLflow, I tracked explanation accuracy, consistency, and alignment with the underlying fraud model outputs. Accuracy measured whether the explanation reflected the correct fraud signals. Consistency ensured similar transactions received similar explanations. Alignment ensured explanations matched the actual model decision.

I then conducted controlled A/B testing, where one group of fraud analysts used traditional explanations and another group used LLM-generated explanations. Analyst decision accuracy improved by 18%, and average fraud investigation time reduced by 25–35% when LLM explanations were used.

I also incorporated qualitative feedback from Fraud Operations teams to refine explanation clarity and usefulness. Combining quantitative ML metrics with qualitative analyst feedback ensured both the fraud models and explanations worked together effectively and delivered measurable business impact.

6. How did you deploy the solution at scale without impacting transaction latency?
Deploying the solution at scale without affecting payment authorization latency was one of the most important technical challenges. I designed a serverless architecture using AWS Lambda and API Gateway to handle LLM inference and explanation delivery efficiently. Fraud scoring and explanation generation were decoupled from the payment authorization flow. The fraud model produced a risk score in real time, while the explanation was generated asynchronously so that legitimate payments were not delayed. This ensured that the system could support millions of transactions per day without performance degradation.

The explainability data layer in Snowflake provided low-latency access to pre-computed features, eliminating expensive runtime calculations. I optimized LLM prompts to be concise, which reduced inference time and cost. API Gateway handled traffic routing and throttling, ensuring system stability during peak transaction volumes. This architecture allowed near real-time fraud explanations to be delivered to internal risk tools while maintaining strict latency SLAs. As a result, PayPal achieved scalable, explainable fraud detection without impacting customer payment experience.

7. How did you handle class imbalance in fraud data?

Handling class imbalance was a critical part of this fraud detection project because fraudulent transactions represented only a very small percentage of total transactions, while genuine transactions made up the majority. If not handled correctly, the model could become biased toward predicting all transactions as non-fraud, which would look accurate on paper but fail in real-world fraud detection.

To address this, I first used SQL on Snowflake to analyze the class distribution and understand fraud frequency across different transaction segments such as geography, device type, and merchant category. This helped identify where fraud was more concentrated. During model training in Python, I applied class weighting so that fraud cases were given higher importance compared to non-fraud cases. This ensured the model paid more attention to learning fraud patterns.

I also used stratified sampling during train-test splits to preserve the original fraud-to-non-fraud ratio. Instead of relying on accuracy, I evaluated models using recall, precision, and precision-recall curves, which are more meaningful for imbalanced datasets. The primary focus was on improving recall, as missing fraud cases has higher business impact.

Additionally, I optimized decision thresholds instead of using a default 0.5 cutoff. Thresholds were tuned based on business risk tolerance, allowing the model to catch more fraud while keeping false positives stable. This approach contributed directly to the 12–18% improvement in fraud recall without degrading customer experience.

8. How did you collaborate with Fraud Operations and Compliance teams?

Collaboration with Fraud Operations and Compliance teams was essential to ensure that the models and explanations were practical, trusted, and compliant with regulations. Early in the project, I worked closely with Fraud Operations analysts to understand how they review cases, what signals they trust, and where they face challenges during investigations.

Based on their feedback, I tailored both the fraud models and the LLM-generated explanations to highlight the most relevant signals, such as transaction velocity, device anomalies, and unusual spending behavior. I conducted review sessions where analysts evaluated sample explanations and flagged areas that were unclear or misleading. This feedback loop helped reduce analyst clarification requests by 22 percent.

For Compliance teams, the focus was on explainability and regulatory alignment. I ensured that explanations avoided technical jargon and clearly mapped to observable transaction behavior. I worked with Compliance to define acceptable explanation formats and terminology that met regulatory expectations.

The explanations were then integrated into analyst dashboards using Tableau, making them easily accessible during case reviews. This collaboration reduced average fraud investigation time by 25–35 percent and improved analyst confidence in decision-making. Overall, close cross-functional collaboration ensured that the solution delivered real business value beyond model performance.

9. How did you monitor model and explanation drift over time?
Monitoring drift was critical because fraud patterns constantly evolve, and both ML models and LLM explanations can degrade over time if not monitored. For fraud models, I tracked performance metrics such as recall, precision, and false positive rate on recent transaction data using scheduled evaluations. For the LLM-generated explanations, I implemented monitoring using MLflow, where I logged metrics related to explanation accuracy, consistency, and alignment with the underlying fraud scores. For example, I checked whether explanations consistently referenced the strongest fraud signals and whether similar transactions produced similar explanations.

I also ran periodic A/B testing, comparing newer prompt versions against baseline prompts. Any increase in misleading or inconsistent explanations triggered prompt refinement or additional constraints. During this process, explanation errors were reduced by 30 percent through controlled testing and iteration. Additionally, I worked with Fraud Operations teams to collect qualitative feedback on explanation quality. Analyst feedback was treated as a signal for drift detection. If analysts started questioning explanations more frequently, it indicated potential misalignment. This combined quantitative and qualitative monitoring approach ensured long-term stability and trust in both fraud scoring and explanation systems.

10. What was the biggest technical challenge and how did you solve it?
The biggest technical challenge in this project was balancing real-time performance, explainability, and scale. Fraud detection systems operate under strict latency constraints, and adding Generative AI explanations introduced the risk of slowing down payment authorization.
To solve this, I designed the architecture so that fraud scoring and explanations were decoupled. Fraud models produced risk scores immediately, while explanations were generated asynchronously using AWS Lambda and API Gateway. This ensured that payment authorization latency was not impacted
I also built an optimized explainability data layer in Snowflake, where high-cardinality features were pre-aggregated into inference-ready tables. This eliminated the need for expensive real-time computations and supported millisecond-level SLA compliance.
Prompt optimization was another key solution. I kept prompts short, structured, and strictly grounded in data to ensure fast LLM inference. This allowed the system to scale and support millions of transactions per day.
By combining efficient data design, scalable AWS infrastructure, and careful prompt engineering, I successfully delivered a system that improved fraud recall, reduced investigation time by 25–35 percent, and met strict performance and compliance requirements.

Behavioral Interview Questions & Answers

1. Tell me about a time you solved a complex problem using data.
In my previous role, I worked on a fraud detection project where the main challenge was improving fraud detection accuracy without increasing false positives. Fraud data is highly imbalanced, and small mistakes can impact genuine customers. I started by deeply analyzing historical transaction and behavioral data using SQL on Snowflake to identify patterns that were strongly correlated with fraud, such as rapid transaction velocity and unusual device behavior.

I then built and optimized machine learning models using Python and focused on recall and precision instead of accuracy. Through careful feature engineering and threshold tuning, fraud recall improved by 12–18 percent. However, the real complexity came when fraud analysts struggled to understand why transactions were flagged.

To address this, I designed a Generative AI–based explanation system using AWS Bedrock that translated model outputs into simple explanations. This helped analysts make faster and more accurate decisions. As a result, analyst decision accuracy improved by 18 percent, and investigation time reduced by 25–35 percent.

This experience showed how combining strong data analysis with practical business understanding can solve complex problems effectively.

2. Describe a time when you had to work with non-technical stakeholders.
I worked closely with Fraud Operations and Compliance teams, who were non-technical but critical stakeholders. Initially, there was a gap between how data scientists viewed model outputs and how analysts interpreted them. I addressed this by conducting regular walkthrough sessions where I explained fraud signals in simple language and gathered feedback on what information analysts needed.

Based on their input, I redesigned fraud explanations to focus on clear behavioral patterns instead of technical metrics. This collaboration reduced analyst clarification requests by 22 percent. I also worked with Compliance teams to ensure explanations met regulatory standards.

By translating technical concepts into business language, we built trust and delivered a solution that improved efficiency and decision accuracy.

3. Tell me about a time you failed or faced a major challenge. How did you handle it?
One major challenge I faced during the project was related to LLM-generated fraud explanations in the early stages of development. While the Generative AI system was producing fluent explanations, we noticed that in some edge cases, the explanations included assumptions that were not fully supported by the underlying data or fraud model outputs. This was a serious concern, especially in a regulated financial environment, because misleading explanations could affect analyst decisions and compliance obligations. Instead of pushing the solution into production quickly, I treated this as a failure point and paused the rollout. I first worked to understand the root cause and realized that the prompts were too flexible and allowed the model to infer beyond the provided inputs. To fix this, I redesigned the prompt-engineering pipeline in Python to strictly ground the LLM on verified inputs such as fraud scores, transaction velocity, device indicators, and customer behavior aggregates.

I also introduced evaluation and hallucination-control metrics using MLflow, where I tracked explanation accuracy, consistency, and alignment with the fraud models. We then ran controlled A/B testing, comparing earlier versions of explanations with refined versions. Through multiple iterations, we reduced incorrect or misleading explanations by 30%. This experience taught me the importance of responsibility and caution when deploying Generative AI systems. It reinforced that in high-impact domains like payments and fraud, correctness and trust are more important than speed. The challenge ultimately helped improve the quality, safety, and credibility of the final solution.

4. How do you prioritize when working on multiple high-impact tasks?
When working on multiple high-impact tasks, I prioritize based on business risk, customer impact, and system dependencies. In the fraud detection project, I was simultaneously responsible for improving fraud recall, building explainability features, and supporting deployment at scale. Trying to do everything at once would have increased risk, so clear prioritization was essential.

My first priority was always the core fraud detection models, because incorrect fraud decisions directly affect customers and financial losses. I focused on stabilizing model performance, improving recall by 12–18 percent, and ensuring false positives remained under control. Only once the models were reliable did I move significant effort toward Generative AI explanations.

I used sprint planning and milestone-based execution to break work into manageable phases. For example, explainability features were initially tested in offline and internal analyst environments before being integrated into real-time workflows. This reduced risk and allowed feedback without impacting live transactions.

I also regularly aligned priorities with stakeholders from Fraud Operations and Compliance. If analysts were experiencing investigation delays, explanation clarity became a higher priority. If performance or latency risks surfaced, infrastructure optimization on AWS and Snowflake took precedence. By continuously reassessing priorities based on impact and urgency, I was able to deliver improvements in fraud detection accuracy, explanation quality, and analyst efficiency without delaying production timelines or compromising system stability.

5. Describe a time you had to defend your technical decision.

One situation where I had to strongly defend a technical decision was the use of Generative AI for fraud explanations. Some stakeholders, especially from Compliance and Risk teams, were initially skeptical about using LLMs due to concerns around hallucinations, regulatory risk, and explainability. Instead of arguing from a purely technical standpoint, I focused on data and evidence. I explained the architecture clearly, showing that the LLM was not making fraud decisions but only explaining outputs from existing fraud models. I highlighted the strict prompt grounding approach, where explanations were generated only from verified model scores and transaction features.

To further support the decision, I presented results from controlled A/B testing. These results showed an 18 percent improvement in fraud analyst decision accuracy, a 22 percent reduction in clarification requests, and a 25–35 percent reduction in investigation time when LLM-generated explanations were used. I also demonstrated that hallucination-control metrics tracked through MLflow reduced misleading explanations by 30 percent. I worked closely with Compliance to ensure explanation formats met regulatory standards and avoided speculative language. This collaborative and data-driven approach helped address concerns and build trust.

Ultimately, the stakeholders approved the solution because it was backed by measurable improvements, clear safeguards, and strong governance. This experience reinforced the importance of defending technical decisions with evidence, transparency, and cross-functional alignment rather than opinion alone.

ACCENTURE, INDIA

Project Overview

At Accenture, I worked as a Data Scientist on a large-scale retail demand forecasting and supply chain optimization project for a global retail client. The main goal of the project was to accurately predict product demand so that the business could reduce stock-outs, avoid over-stocking, and improve overall inventory planning across stores and regions. The system handled demand forecasting for 60,000+ SKUs across more than 1,200 stores, making it a high-volume, real-world production use case.

My responsibility was to build, improve, and deploy machine learning forecasting models. I used statistical models like ARIMA, SARIMA, and Prophet for baseline forecasts and advanced ML models such as CatBoost and Gradient Boosting for complex demand patterns. By engineering features from historical sales, promotions, holidays, competitor pricing, and macroeconomic indicators, we improved forecast accuracy by around 23% compared to the previous rule-based system. For high-volatility and seasonal products, probabilistic and boosting models helped reduce peak-season forecast errors by 27%.

I also designed hierarchical forecasting (SKU → store → region), which aligned predictions with business planning and improved regional forecast accuracy by 19%. On the data side, I built automated ETL and batch inference pipelines using Python, SQL, and Apache Airflow, processing over 100 million records per day. For reliability, I implemented model monitoring and anomaly detection using Isolation Forest, z-score checks, and control charts, which reduced forecast failures by 34%. For deployment, models were exposed through FastAPI services and Docker containers, allowing easy integration with inventory and supply chain systems. I also created Tableau dashboards to show forecast vs actuals, seasonality trends, and KPIs for leadership. Overall, this project delivered measurable business impact and gave me strong experience in end-to-end production-grade data science.

Detailed Step by step Explanation of your Project

Step 1: Understanding the Business Problem and Data Landscape

The first thing I worked on as a Data Scientist at Accenture was understanding the business problem and the scale of data. The client was a large retail organization that wanted to improve demand forecasting accuracy to reduce stock-outs, excess inventory, and supply chain inefficiencies. The system needed to forecast demand for 60,000+ SKUs across 1,200+ stores, covering multiple regions and product categories. Existing forecasts were mostly rule-based and were failing during promotions, holidays, and peak seasons.

I collaborated closely with business stakeholders, supply chain teams, and merchandising teams to understand how forecasts were used for purchasing, replenishment, and regional planning. From these discussions, I identified key forecasting frequencies such as daily and weekly forecasts, and understood which products had high volatility, seasonal spikes, or irregular demand patterns. This helped define clear success metrics like forecast accuracy improvement, reduction in peak-season errors, and system reliability.

On the data side, I analyzed large datasets including historical sales data, promotions, holiday calendars, competitor pricing, and macroeconomic indicators. The raw data volume was massive—over 100 million records processed per day. I used SQL to profile data quality, check missing values, outliers, and inconsistencies at SKU and store levels. This step helped me identify data gaps, demand spikes, and noisy SKUs early in the process.

By the end of this step, I had a clear understanding of what to predict, at what level, how often, and why it mattered, which laid a strong foundation for building accurate and scalable forecasting models.

Step 2: Data Preparation, Feature Engineering, and Data Pipelines

Once the problem was clear, my next responsibility was to prepare clean, reliable, and model-ready data. I built automated ETL pipelines using Python, SQL, and Apache Airflow to ingest data from multiple sources. These pipelines processed 100M+ records per day, ensuring fresh data availability for daily and weekly forecasts. Airflow DAGs were scheduled to handle extraction, transformation, validation, and loading with proper failure handling and retries.

Feature engineering was a major part of my role. I created meaningful features from historical sales trends, rolling averages, lag variables, promotional flags, holiday indicators, price changes, and competitor pricing signals. For example, I generated 7-day, 14-day, and 30-day rolling sales features to capture short-term and long-term demand behavior. Holiday and promotion features helped models understand sudden demand spikes.

To manage high-dimensional data, I applied feature selection techniques such as mutual information to measure feature importance and PCA (Principal Component Analysis) to reduce dimensionality without losing critical demand signals. This improved model efficiency and reduced overfitting, especially for low-volume SKUs.

I also handled missing values, extreme outliers, and inconsistent pricing data using statistical methods and business rules. This step significantly improved data quality and ensured stable model training. Strong data preparation directly contributed to the 23% improvement in forecast accuracy achieved later in the project.

Step 3: Model Development Using Statistical and Machine Learning Techniques

In this step, my primary responsibility was to build and train demand forecasting models using both statistical and machine learning approaches. I started with classical time-series models such as ARIMA and SARIMA to capture trend and seasonality for stable SKUs. These models were useful for products with consistent historical patterns and helped establish strong baseline forecasts.

Next, I implemented Prophet for SKUs affected by holidays and yearly seasonality. Prophet allowed me to explicitly include holiday calendars and automatically handle trend changes, which improved forecast stability during festive periods. These statistical models provided interpretable results and were useful for business validation.

For more complex demand behavior, I developed machine learning models using CatBoost and Gradient Boosting. These models were particularly effective in handling non-linear relationships between demand and features such as promotions, competitor pricing, and macroeconomic signals. CatBoost also handled categorical variables efficiently without extensive preprocessing.

By comparing model performance across SKUs, I selected the best model type based on volatility and data availability. This hybrid modeling approach helped improve overall forecast accuracy by 23% compared to the previous system. I evaluated models using time-based validation and metrics like MAPE and RMSE to ensure realistic performance.

This step ensured that each SKU received the most suitable forecasting method, balancing accuracy, scalability, and interpretability across 60,000+ products.

Step 4: Hierarchical Forecasting and Business Alignment

After developing individual models, I focused on implementing hierarchical forecasting to align predictions across different business levels. The client required forecasts at SKU, store, and regional levels, and these numbers had to be consistent for planning and decision-making.

I designed a forecasting framework where predictions were generated at the SKU → Store → Region hierarchy. This meant ensuring that SKU-level forecasts correctly aggregated to store-level totals and that store forecasts rolled up accurately to regional demand. This alignment was critical for procurement, replenishment, and merchandising teams.

To achieve this, I applied hierarchical reconciliation techniques that adjusted lower-level forecasts to match higher-level constraints without losing SKU-level accuracy. This helped avoid situations where regional demand numbers conflicted with individual store or SKU forecasts.

As a result of this structured approach, regional forecast accuracy improved by 19%, which directly supported better inventory allocation and regional planning. Leadership teams were able to confidently use forecasts for budgeting and supply chain decisions.

This step significantly increased business trust in the models, as all stakeholders—from store managers to regional planners—worked with a single, consistent version of demand forecasts.

Step 5: Monitoring, Anomaly Detection, and Model Stability

After deploying the demand forecasting models, my responsibility was to make sure they remained accurate, stable, and trustworthy over time. Retail demand data changes frequently due to promotions, holidays, pricing changes, and external factors, so continuous monitoring was critical. I designed a model monitoring and anomaly detection framework to identify issues early before they impacted business decisions.

I implemented Isolation Forest models to automatically detect unusual demand patterns at the SKU and store levels. This approach helped identify sudden demand spikes or drops that did not match historical behavior. In parallel, I used z-score–based statistical checks to measure how far current demand deviated from historical averages. When deviations crossed defined thresholds, alerts were triggered.

To add an additional layer of control, I applied statistical control charts, which tracked demand behavior over time and flagged abnormal trends. These techniques helped detect data drift, model drift, and input data quality issues. For example, if promotional data was missing or pricing signals changed abruptly, the system detected it immediately.

Monitoring was performed on daily and weekly forecasts, covering 60,000+ SKUs across 1,200+ stores. Alerts were integrated into the Airflow pipelines so failures could be investigated quickly. Based on detected issues, models were either retrained, recalibrated, or temporarily overridden with fallback forecasts.

This proactive monitoring approach reduced forecast failures by 34%, significantly improving system reliability during peak seasons. It also increased confidence among supply chain and merchandising teams, as they could rely on forecasts even during volatile demand periods. Overall, this step ensured that the forecasting system remained production-grade, resilient, and scalable.

Step 6: Deployment, APIs, Dashboards, and Business Impact

In the final stage of the project, I focused on making sure the models were easily accessible, scalable, and usable by business teams. I deployed the forecasting models using Docker, which ensured consistent environments across development, testing, and production. Each model was exposed through FastAPI-based APIs, allowing supply chain and inventory systems to request forecasts programmatically.

I designed batch inference workflows to generate daily and weekly forecasts automatically. These pipelines were orchestrated using Apache Airflow and processed 100M+ records per day without manual intervention. Forecast outputs were stored in centralized databases and consumed by downstream planning tools.

To support business users, I developed interactive Tableau dashboards that visualized forecast vs actuals, error trends, seasonality patterns, and category-level KPIs. Leadership teams used these dashboards to monitor performance across regions, while merchandising teams analyzed SKU-level trends for promotions and inventory decisions.

The dashboards also highlighted key metrics such as 23% improvement in forecast accuracy, 27% reduction in peak-season forecast errors, and 19% improvement in regional forecasts. This made the business value of the models easy to understand for non-technical stakeholders.

As a result of this end-to-end deployment, the client experienced fewer stock-outs, reduced excess inventory, and faster decision-making. The automated and scalable system significantly reduced manual forecasting effort and improved planning efficiency.

This step demonstrated my ability to translate complex machine learning models into real business impact, manage large-scale production systems, and communicate results clearly—core responsibilities of a Data Scientist at Accenture.

Technical Interview Questions & Answers

1. How did you approach demand forecasting for 60,000+ SKUs across 1,200+ stores?

I approached demand forecasting by first understanding that not all SKUs behave the same. Some products have stable demand, while others are highly seasonal or promotion-driven. Because of this, I used a hybrid forecasting strategy rather than a single model for all products.

I grouped SKUs based on demand patterns such as volume, volatility, and seasonality. For stable SKUs, I used statistical time-series models like ARIMA and SARIMA, which are effective at capturing trend and seasonality. For products affected by holidays and yearly cycles, I used Prophet, as it handles holiday effects very well.

For complex and volatile SKUs, I used machine learning models like CatBoost and Gradient Boosting, which can learn non-linear relationships from multiple features such as promotions, pricing, and external signals.

Forecasts were generated at SKU, store, and regional levels using hierarchical forecasting to keep numbers consistent. This approach improved overall forecast accuracy by 23% and regional accuracy by 19%, while also scaling efficiently across 60,000+ SKUs.

2. Why did you use CatBoost instead of other ML models for demand forecasting?

I chose CatBoost mainly because of its ability to handle categorical features and complex non-linear relationships very efficiently. In retail demand forecasting, features like product category, store type, region, and promotion flags are extremely important. CatBoost handles these categorical variables internally without extensive one-hot encoding, which helps reduce feature explosion and training time.

Another reason was CatBoost's robustness to noisy data, which is very common in large retail datasets. Since we were processing 100M+ records per day, data quality issues like missing values and outliers were unavoidable. CatBoost handled these scenarios better compared to traditional boosting models.

CatBoost also reduced overfitting through ordered boosting, which was important when modeling high-volatility SKUs. When compared to baseline statistical models, CatBoost significantly improved performance for promotion-driven and peak-season products, contributing to a 27% reduction in peak-season forecast errors. Overall, CatBoost provided a good balance of accuracy, scalability, and interpretability, making it suitable for production-level forecasting systems.

3. How did you perform feature engineering for demand forecasting models?

Feature engineering was one of the most critical parts of the project. I created features from multiple data sources, including historical sales, promotions, holidays, competitor pricing, and macroeconomic indicators.

From sales data, I generated lag features (such as sales from the previous 7, 14, and 30 days) and rolling averages to capture short-term and long-term demand trends. These helped the models understand momentum and seasonality.

I added promotion flags, discount depth, and holiday indicators to capture sudden demand spikes. Pricing features included relative price changes and competitor price differences to explain demand shifts caused by market competition.

To manage high dimensionality across 60,000+ SKUs, I used mutual information to measure feature importance and remove weak predictors. I also applied PCA to reduce dimensionality while preserving important demand signals, improving model efficiency and training stability.

This structured feature engineering directly contributed to a 23% improvement in forecast accuracy.

4. How did you handle high-volatility and seasonal SKUs differently?

High-volatility SKUs required special treatment because traditional forecasting models often fail during peak seasons or promotions. I first identified these SKUs using statistical measures like variance, coefficient of variation, and historical error patterns.

For such products, I relied more on machine learning and probabilistic forecasting techniques instead of purely statistical models. Gradient boosting models helped capture complex interactions between promotions, pricing, and external signals. I also used probabilistic forecasts to estimate demand ranges instead of single-point predictions. This helped supply chain teams plan safety stock more effectively. During peak seasons, this approach reduced forecast errors by 27%, significantly improving inventory availability. Additionally, monitoring thresholds for volatile SKUs were stricter, ensuring anomalies were detected quickly.

5. Explain the hierarchical forecasting approach you used.

Hierarchical forecasting was required because forecasts were needed at multiple business levels—SKU, store, and region—and all these numbers had to match. If SKU-level forecasts did not add up to regional numbers, it created confusion in supply chain and merchandising planning.

I started by generating forecasts at the most granular level, which was SKU-store level, because this level captures the most detailed demand patterns. These forecasts were created using statistical and ML models depending on SKU behavior. After that, I aggregated the forecasts upward to store and regional levels.

To ensure consistency, I applied forecast reconciliation techniques, where lower-level forecasts were adjusted so that their sums exactly matched higher-level forecasts. This helped remove conflicts between SKU, store, and regional numbers.

This approach allowed different teams to use the same forecasts confidently. Store teams focused on SKU forecasts, while leadership looked at regional demand. Because of this alignment, regional forecast accuracy improved by 19%, and planning errors reduced significantly. Hierarchical forecasting also increased business trust, as all teams worked from a single source of truth.

6. How did you build and automate ETL pipelines for this project?

I built fully automated ETL pipelines using Python, SQL, and Apache Airflow to handle very large datasets. These pipelines processed 100M+ records per day, pulling data from multiple systems such as sales databases, promotion calendars, pricing feeds, and external macroeconomic sources.

In Airflow, I designed DAGs that controlled each step—data extraction, transformation, validation, and loading. SQL was used for joins, aggregations, and filtering, while Python handled complex transformations and feature preparation.

I added data quality checks at each stage, such as missing value detection, schema validation, and outlier checks. If a job failed, Airflow automatically retried it and raised alerts. This ensured fresh, reliable data was always available for daily and weekly forecasting.

Because of automation, manual effort was almost eliminated, and forecasting pipelines ran consistently at scale across 60,000+ SKUs and 1,200+ stores.

7. How did you monitor model performance and detect anomalies?

After deployment, I focused heavily on model monitoring, because demand patterns in retail change frequently. I implemented a combination of ML-based and statistical techniques to detect issues early.

I used Isolation Forest to identify abnormal demand patterns such as sudden spikes or drops at SKU and store levels. This was especially useful for detecting unusual behavior during promotions or data issues.

Along with this, I used z-score analysis to measure how far current demand deviated from historical averages. I also applied control charts to track demand trends over time and detect structural changes.

These monitoring checks ran on daily and weekly forecasts. When anomalies were detected, alerts were triggered and affected SKUs were reviewed or retrained. This approach helped identify data drift and model drift early. As a result, forecast failures reduced by 34%, and the system remained stable even during high-demand seasons.

8. How did you deploy your models into production?

To deploy the demand forecasting models into production, I focused on making the system scalable, reliable, and easy to integrate with existing supply chain applications. I started by containerizing all models using Docker, which ensured that the same environment was used across development, testing, and production. This eliminated dependency and version mismatch issues.

I then exposed the models using FastAPI, creating lightweight REST APIs that allowed downstream systems to request forecasts programmatically. This was important because forecasts needed to be consumed by inventory planning and replenishment systems across 1,200+ stores.

For large-scale forecasting, I implemented batch inference pipelines that generated daily and weekly forecasts automatically. These jobs were orchestrated using Apache Airflow, which handled scheduling, retries, and

failure alerts. The pipelines processed 100M+ records per day, making them suitable for enterprise-scale workloads.

Forecast outputs were stored in centralized databases and versioned to support audits and backtracking. This deployment approach ensured high availability, reduced manual intervention, and allowed models to run reliably during peak demand periods.

9. How did you evaluate forecasting model performance?

Evaluating forecasting models correctly was critical to ensure real business impact. Since this was time-series data, I avoided random data splits and instead used time-based cross-validation. Models were trained on historical periods and tested on future time windows to simulate real-world performance.

I used MAPE and RMSE as primary evaluation metrics because they are easy to interpret for both technical and non-technical stakeholders. Evaluation was performed at multiple levels—SKU, store, and region—to ensure models performed well both at granular and aggregated views.

I also compared performance across different SKU groups, such as stable, seasonal, and high-volatility products. This helped me select the most suitable model type for each SKU category. For example, ML models performed better for promotion-driven SKUs, while statistical models worked well for stable items.

This structured evaluation process helped achieve a 23% improvement in forecast accuracy and ensured consistent performance across 60,000+ SKUs.

10. How did you communicate results to non-technical stakeholders?

Communicating results clearly was a key responsibility, especially since many stakeholders were non-technical. I focused on translating complex model outputs into simple business insights. I built interactive Tableau dashboards that displayed forecast vs actuals, error trends, seasonality patterns, and category-level KPIs.

Instead of discussing algorithms, I highlighted measurable outcomes such as 23% improvement in forecast accuracy, 27% reduction in peak-season errors, and 19% improvement in regional forecasts. These metrics helped leadership quickly understand the value delivered by the models.

I also used visual examples to explain why forecasts changed during promotions or holidays. This helped build trust and increased adoption among supply chain and merchandising teams. Regular review sessions were conducted where stakeholders could ask questions and provide feedback. This collaborative approach ensured forecasts were actively used in decision-making and not treated as a black box.

Behavioral Questions & Anwers

1. Tell me about a challenging data science project you worked on and how you handled it.
Answer:
One of the most challenging projects I worked on was a large-scale retail demand forecasting system covering 60,000+ SKUs across 1,200+ stores. The main challenge was handling highly volatile demand caused by promotions, holidays, and regional differences. Traditional forecasting approaches were failing during peak seasons, leading to stock-outs and excess inventory.

I approached this by first segmenting SKUs based on volatility and demand behavior. For stable products, I used statistical models like ARIMA and SARIMA, while for complex and promotion-driven SKUs, I implemented machine learning models such as CatBoost. I also engineered features from promotions, holidays, and competitor pricing to better capture demand drivers.

Another challenge was ensuring forecasts aligned across SKU, store, and regional levels. I solved this using hierarchical forecasting, which improved regional forecast accuracy by 19%.

As a result, the project achieved a 23% improvement in forecast accuracy and 27% reduction in peak-season errors. This experience taught me how to balance technical depth with business impact under real-world constraints.

2. Describe a time when your model did not perform as expected. What did you do?
Answer:
During the early phase of deployment, I noticed that some high-volatility SKUs were producing inaccurate forecasts during promotional periods. Even though offline validation metrics looked strong, real-world performance dropped sharply during peak demand windows.

I first analyzed forecast vs actual trends using Tableau dashboards and identified that the issue was caused by sudden changes in promotion depth and missing external signals. I responded by adding promotion intensity features and holiday proximity variables into the model.

I also implemented model monitoring using Isolation Forest and z-score analysis to detect such anomalies early. For volatile SKUs, I introduced probabilistic forecasting to provide demand ranges instead of point estimates.

After these changes, peak-season forecast errors reduced by 27%, and the monitoring framework helped reduce forecast failures by 34%. This experience taught me the importance of continuous monitoring and adapting models post-deployment.

3. Tell me about a time you had to explain a complex model to non-technical stakeholders.
Answer:
In my demand forecasting project, I worked closely with supply chain managers, merchandising teams, and senior leadership, many of whom did not have a technical background. The challenge was that the forecasting system used advanced models like CatBoost, ARIMA, and hierarchical forecasting, which could easily confuse non-technical stakeholders if explained in technical terms.

To address this, I focused on business storytelling instead of algorithms. Rather than explaining how CatBoost works internally, I explained why the forecast changed and what it meant for inventory and planning decisions. I built Tableau dashboards that showed forecast vs actual demand, seasonal patterns, and error trends using simple visuals.

During review meetings, I used real examples, such as explaining how promotions or holidays caused demand spikes for specific SKUs. I also highlighted measurable outcomes like 23% improvement in forecast accuracy and 19% improvement in regional planning, which helped leadership quickly see the value.

When stakeholders had concerns about trust and reliability, I explained the monitoring framework in simple terms, showing how anomalies were detected early. This approach improved adoption significantly. Over time, stakeholders became comfortable using forecasts in daily decision-making, which demonstrated the importance of clear communication and aligning technical work with business understanding.

4. Describe a situation where you had to work with messy or incomplete data.
Answer:
In this project, I dealt with very large-scale retail data, processing over 100M+ records per day from multiple systems. As expected, the data was often messy, with missing sales values, inconsistent promotion information, and incorrect pricing data. This posed a major risk to forecast accuracy if not handled properly.

My first step was to identify recurring data quality issues by running data profiling using SQL and Python. I built automated checks into the ETL pipelines to detect missing values, schema mismatches, duplicate records, and extreme outliers. These checks ran daily within Apache Airflow, ensuring early detection of issues.

For missing or incorrect data, I applied a combination of business rules and statistical methods. For example, missing sales values were imputed using historical averages or rolling windows, depending on SKU behavior. Pricing inconsistencies were flagged and corrected based on competitor pricing and historical trends.

I also worked closely with upstream data teams to address root causes rather than just fixing symptoms. As data quality improved, model stability increased, directly contributing to a 23% improvement in forecast accuracy.

This experience taught me that strong data engineering and validation are just as important as model building in real-world data science projects.

5. Tell me about a time you had to balance speed and accuracy in a project
Answer
In this retail forecasting project, one of the key challenges was balancing forecast accuracy with strict delivery timelines. The business required daily and weekly forecasts to be available early each day for inventory and replenishment decisions. At the same time, leadership expected significant accuracy improvements over the existing system.

To balance this, I adopted a hybrid modeling strategy. Instead of using complex ML models for every SKU, I segmented products based on demand behavior. For stable SKUs, I used simpler statistical models like ARIMA and SARIMA, which were fast and reliable. For complex, promotion-driven SKUs, I applied machine learning models like CatBoost, where the accuracy gains justified the additional computation.

I also optimized the data pipelines using batch processing and Airflow scheduling, ensuring that 100M+ records per day could be processed efficiently without delays. Feature selection and dimensionality reduction using mutual information and PCA further reduced training and inference time.

As a result, forecasts were delivered on time while still achieving a 23% improvement in accuracy and a 27% reduction in peak-season errors. This experience showed me how to make practical trade-offs between speed and accuracy, which is critical in production data science systems.